

KHULNA UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Final Project: CPU DESIGN

03-08-2025

25 bit

SUBMITTED TO :	SUBMITTED BY :
M.M.A. Hashem Professor , Department of Computer Science & Engineering Khulna University of Engineering & Technology (KUET) **Md. Shakhawat Hossain** Lecturer , Department of Computer Science and Engineering Khulna University of Engineering & Technology (KUET)	Name : Md. Siam Kabir Roll : 2207106 Section : B Department : CSE Date : 03-08-2025

Objectives:

- Understand how the ALU, registers, and other parts form a CPU.
- Learn to build and visualize computer systems using Logisim.
- Explore the architecture and data flow in a computer system.
- Study how data is transferred and processed in real computers.
- Understand CPU functions, including the fetch–execute cycle.
- Learn the roles of MAR, MBR, and the Program Counter.
- Get a clear view of how hardware and software work together.

Introduction:

A computer system is made up of different parts what work together to process data and carry out instructions. These parts usually includes the central processing unit (CPU), memory, input/output devices, and control logic. By building a simple computer in Logisim, we can better understanding how these parts communicate and how instructions is carried out inside a computer.

The 25-bit mini computer I am creating performs basic operations like Addition, Substraction, OR, AND, Shift Right, Shift Left, Store, Load, etc. The CPU is a more complex digital circuit that acts like the brain of the computer. It fetch instructions from memory, decodes and executes them, and controls the flow of data between different components of the system. Below is an overview of the role each component plays within the CPU:

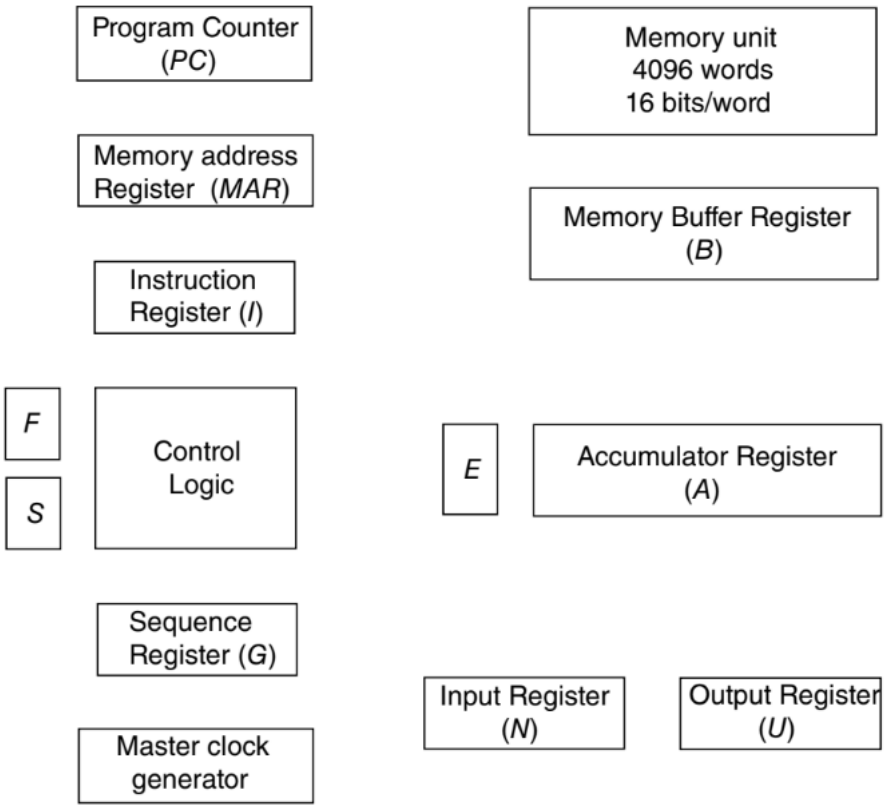
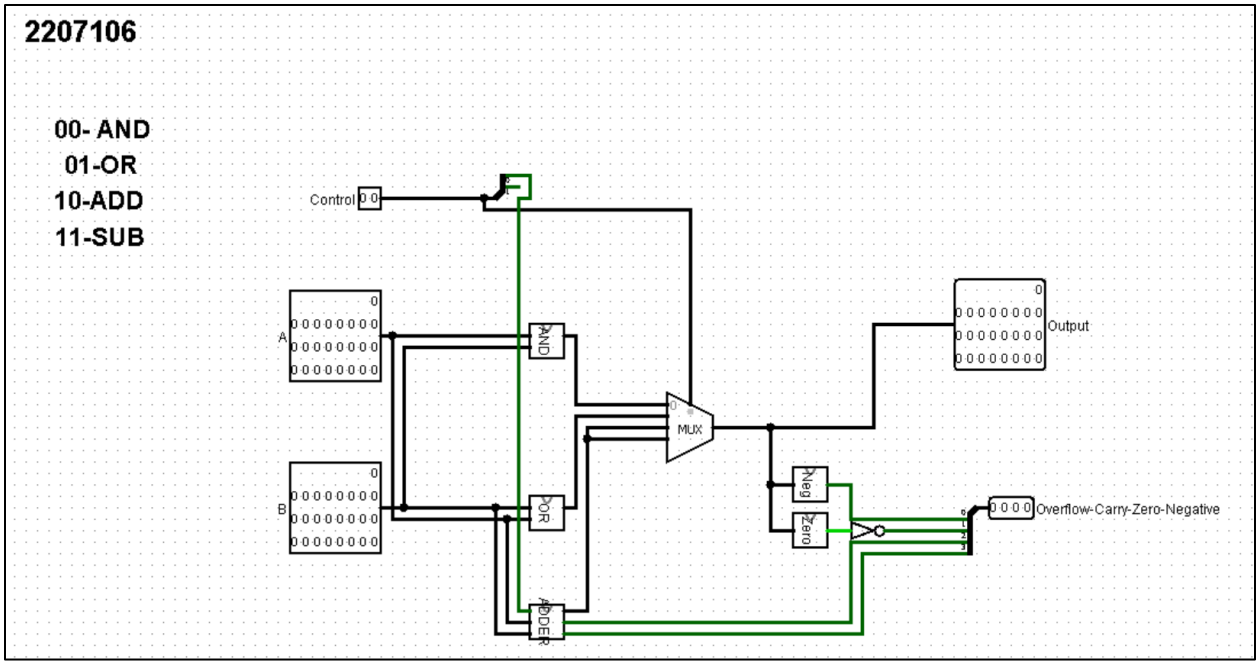


Figure 11.1 Block diagram of digital computer

ALU(Arithmetic Logic Unit):

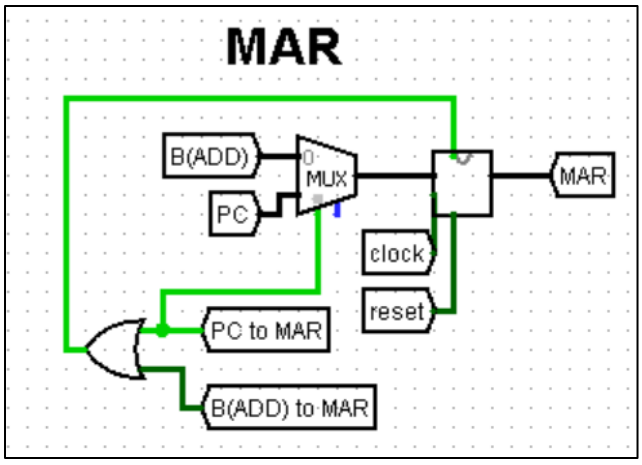
The ALU is the part that does all the math and logic stuff in a computer. It handles arithmetic operations like add, subtract, multiply, and divide, and also logical ones like AND, OR, and NOT. It takes input either from memory or registers, does the operation, then gives back the result.

My ALU, in this mini 25-bit computer, does all these basic things and helps the CPU do what it's supposed to. It plays a big role in running instructions properly.



MAR (Memory Address Register):

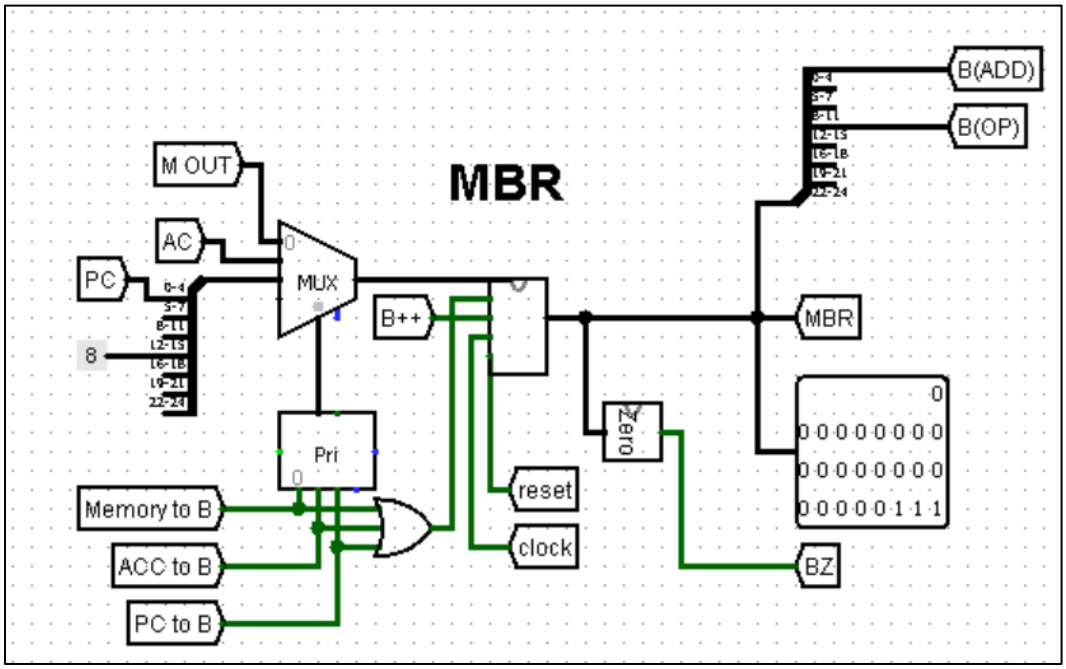
The Memory Address Register, or MAR, is used by the CPU to store the address of the data or instruction that it needs to access from memory. So basically, whenever the CPU wants to read or write something in memory, it first puts the exact memory location into the MAR. Then this address is used to go and fetch the data, or to store something at that place. Without MAR, the CPU wouldn't really know where to go in the memory. It's like giving the memory a home address before doing anything.



MBR (Memory Buffer Register):

The Memory Buffer Register, or MBR, is used to hold the actual data that's being moved in or out of the memory. When the CPU wants to read something from memory, the data first comes into the MBR before going anywhere else. Same way, if the CPU wants to write something into memory, it first puts the data inside the MBR, and then it gets sent to the right place in memory.

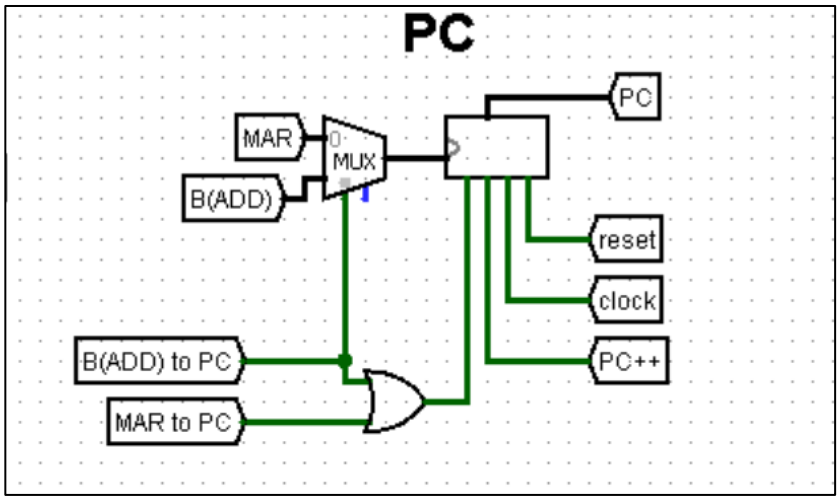
So you can say the MBR works like a temporary holding area or a waiting room for data that's traveling between the memory and the CPU. It helps make sure the data is ready to go, whether it's coming from memory or going into it. Without the MBR, the CPU wouldn't have a smooth way to handle all this data transfer.



Program Counter (PC):

The Program Counter (PC) is a register that holds the address of the next instruction the CPU has to read from memory. It usually increases by 1 after each instruction, so the CPU reads instructions one by one in order. When a program needs to jump somewhere else, like in a loop or function, the PC is updated with that new address, so it knows where to go next.

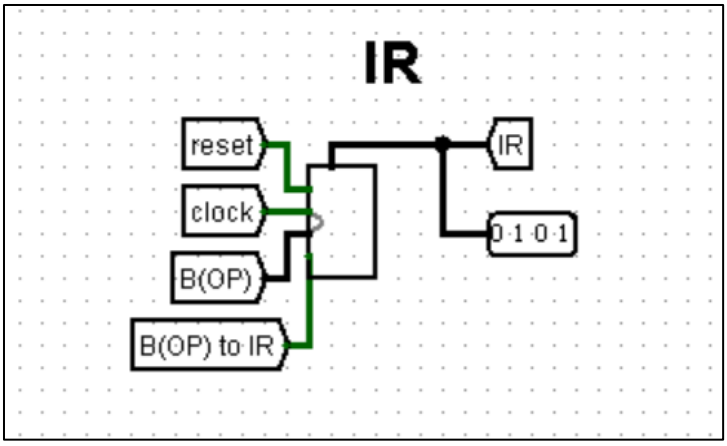
To fetch an instruction, the PC's value is sent to the MAR, and a read operation happens. Because it always keeps track of the next step, the PC helps the processor follow the program properly, whether it's going straight or jumping around.



Instruction Register (IR):

The Instruction Register (IR) holds the operation code (opcode) of the current instruction. It's only 4 bits long since all opcodes in this system are 4 bits. When an instruction is fetched, the opcode part is moved into the I register from the B register, while the address part stays in B.

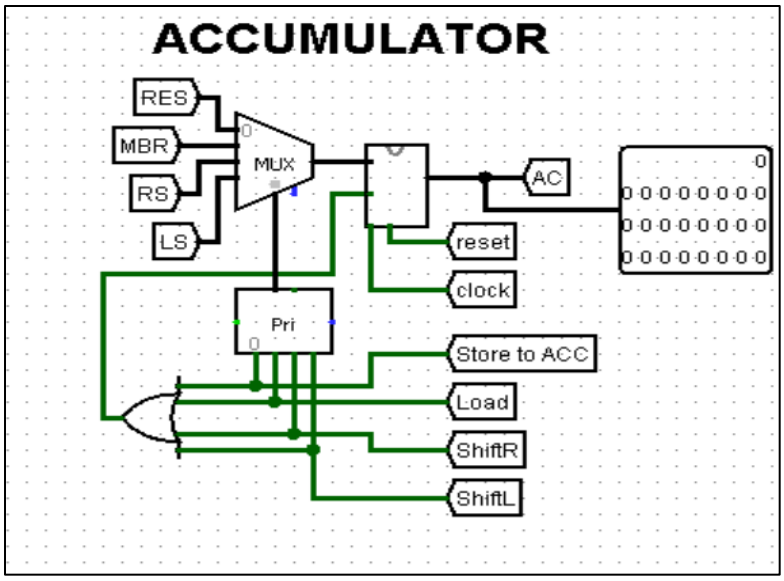
We move the opcode out of B because if we later load some data (an operand) into B, it would erase the instruction stored there before. The opcode in the IR register is important—it tells the control unit what action to take with the data that was just read.



Accumulator (A Register):

The Accumulator, or A register, is a part of the processor that works with data stored earlier in memory. It's used for running most instructions and also for getting data from input devices or sending it to output devices. Together with the B register, it forms the main working part of the processor in this design.

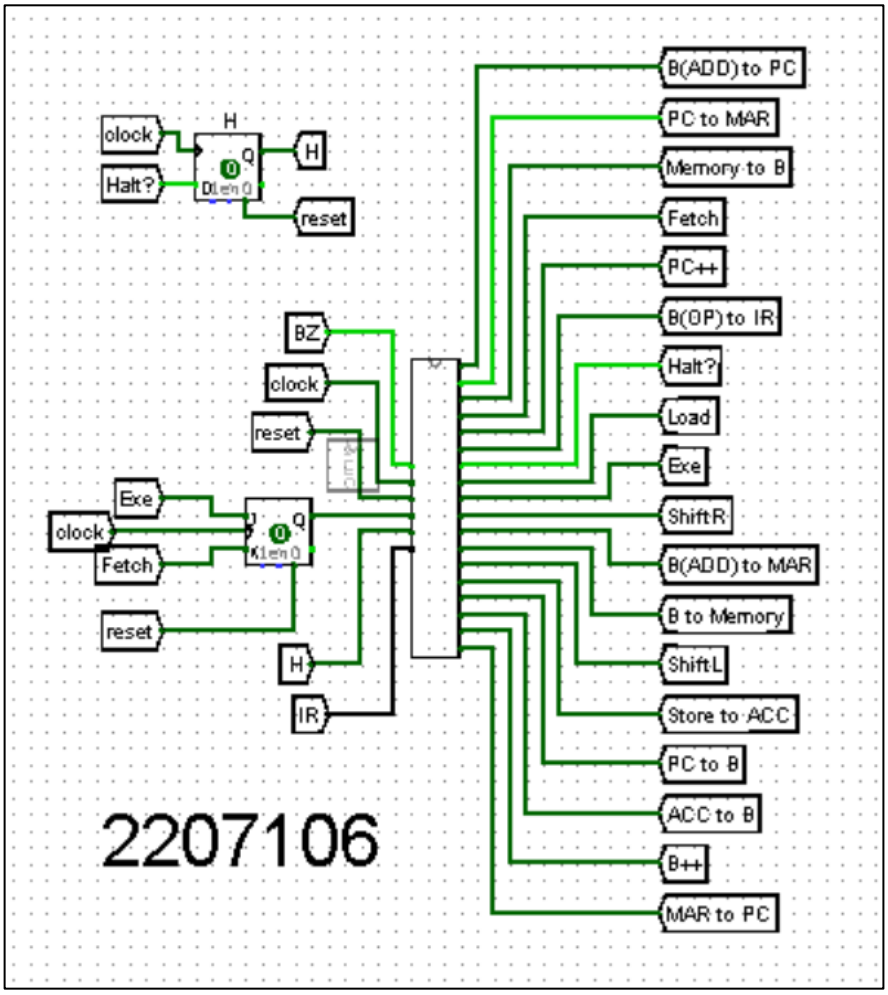
In many systems, there are more registers, but to keep things simple, we're using just one accumulator here. With a single accumulator, we can easily do the add operation. Other operations like subtract, multiply, or divide need to be done using a series of instructions, kind of like a mini subroutine.



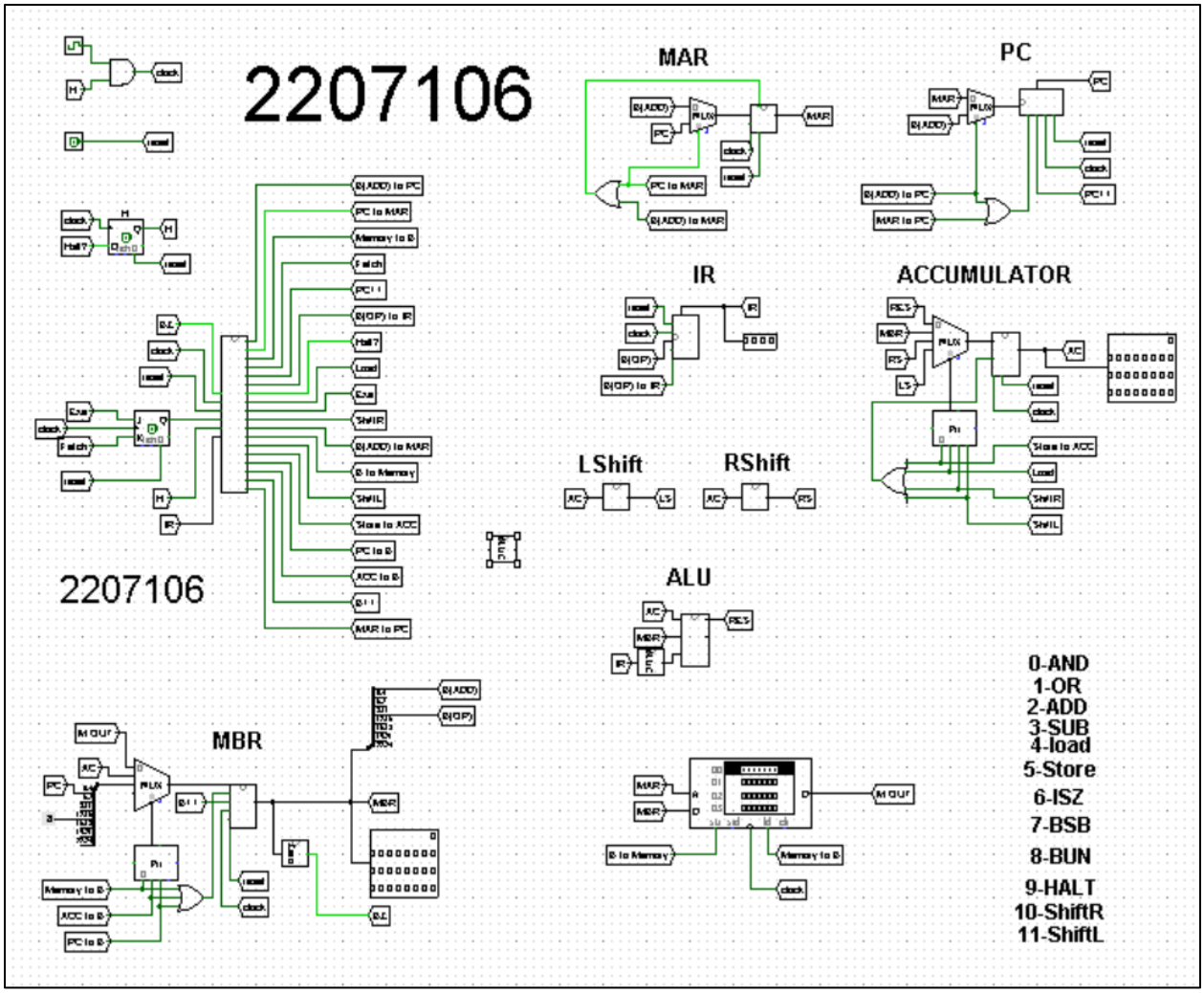
Control Unit:

The Control Unit is like the manager of the CPU. It controls and coordinates everything going on inside. It tells the other parts like the ALU, memory, MAR, MBR, etc., what to do and when to do it by sending control signals.

When an instruction is fetched from memory, the Control Unit reads it and figures out what needs to happen. Then it makes sure all the parts work together—like telling the ALU to do a calculation or asking memory to read or write data. Without the Control Unit, all the other components wouldn't know what to do next.



My CPU:



Operation and Functioning:

In this tiny 25-bit computer, the **instruction** is broken into parts. The **first 5 bits (0 to 4)** hold the **memory address**, and the **opcode** is placed in **bits 12 to 15**. The rest of the bits (in between) can be used for extra control or just left unused, depending on the instruction.

When a **clock pulse** hits, the **Control Unit** starts by **fetching an instruction from RAM**. The **Program Counter (PC)** gives the address, and the fetched instruction goes into the **MBR**.

Let's say the instruction is for an **ADD** operation:

- The **address** part (bits 0–4) is copied from the **MBR** into the **MAR**.
- The **opcode** (bits 12–15) is passed to the **Instruction Register (IR)** and decoded.
- Based on the opcode, the **Control Unit** sends control signals.
- The **ALU** performs the operation, and the **Accumulator** is updated with the result.

Timing Cycle and Control Flow

The CPU works step by step using a timing sequence:
t_0, t_1, t_2, t_3... — each step tells the CPU what to do next.

The Control Unit uses a **4-bit opcode** (from bits 12–15) and decodes it using a **decoder circuit**. It also uses an **F flag** to know which phase it's in:

- **F = 0** → Fetch Cycle
- **F = 1** → Execution Cycle

Opcode Table for 25-bit CPU Instructions

Opcode	Instruction	Description (Optional)
0	AND	Logical AND with Accumulator
1	OR	Logical OR with Accumulator
2	ADD	Add to Accumulator
3	SUB	Subtract from Accumulator
4	LOAD	Load data from Memory to Accumulator
5	STORE	Store Accumulator value to Memory
6	ISZ	Increment Memory and Skip if Zero
7	BSB	Branch and Save Return Address
8	BUN	Unconditional Branch to Address
9	HALT	Stop Program Execution
10	ShiftR	Shift Accumulator Right
11	ShiftL	Shift Accumulator Left

Each instruction follows a **microprogram** stored in the **Instruction Counter (IC)**, which breaks it down into small actions (micro-operations).

CPU Components Used

This CPU is built using the following parts:

- **Registers**: MAR, MBR, PC, IR, Accumulator
- **RAM**
- **Control Unit**
- **Clock**
- **Flip-flops**

Each clock pulse does one job — so the system works in sync and step-by-step.

25-bit Instruction Format Summary

Field Name	Bits Range	Purpose
Extra Control/Data	24–16	Optional for future use or control flags
Opcode	15–12	Defines the operation to perform
Extra Bits	11–5	Can be used for control/data if needed
Address	4–0	Memory location for the instruction

Sample Input:

```
00 000700f 0009000 0000005 0000003 0000000 0000000 0000000 0000000 0000000 0000000 0000000 0000000 0000000 0000000 0000000 0000000
10 0004002 0000003 0005004 0001003 0005005 0002003 0005006 0003003 0005007 000a002 0005008 000b002 0005009 000600a 000500b 000800f
```

BSB > HALT > VALUES > LOAD > AND > STR > OR > STR > ADD > STR >SUB > STR > SHIFTR > STR > SHIFTL > STR > ISZ > STR > BUN

Output:

```
00 000700f 0009000 0000005 0000003 0000001 0000003 0000006 0000003 0000001 0000002 0000001 0000000 0000000 0000000 0000000 0008001
10 0004002 0000003 0005004 0001003 0005005 0002003 0005006 0003003 0005007 000a002 0005008 000b002 0005009 000600a 000500b 000800f
```

Discussion:

From the first lab to the last one, we were introduced to different parts of a computer system. At the beginning, we learned about the **ALU**, then we were given tasks to build and understand its working. Later, we explored **Booth's algorithm** for multiplication, followed by the concept of **cache memory**, and gradually moved on to how a **CPU** functions step by step.

In this final project, I used all those concepts together to design a **25-bit mini computer system**. This system supports a total of **8 main operations**: **ADD, AND, SZI, STORE, BUN, BSB, LOAD, HALT**, along with **4 extra instructions** that I added myself: **SHIFT RIGHT, OR, SUB, and SHIFT LEFT**.

To manage the instruction size and reduce complexity, I used **bit mapping** from binary to hexadecimal. The instruction format was designed to keep **address in bits 0–4**, and **opcode in bits 12–15**, following the custom 25-bit format. After the system was built in **Logisim**, I tested it with sample instructions, and it worked fine as expected—it followed the operations smoothly and step by step.

Conclusion:

In conclusion, the project successfully met its objective of designing and implementing a working **25-bit computer system** using **Logisim**. Through this project, I got hands-on experience with key topics like how a **CPU fetches and executes instructions**, how **data flows** between memory and registers, and how control signals operate within a basic computer architecture.

Even though this system supports a limited set of instructions, it gave a strong base for understanding how a real computer works internally. It helped me understand the whole execution cycle, data transfers, and memory handling in a more practical way. This 25-bit mini computer now serves as a good platform to explore more advanced digital design topics in the future.

References:

1. Digital Logic and Computer Design by M. Morris Mano
2. Class Lecture